

Clean Beauty Recommender Project 4: Business Proposal for Safer & Inclusive Recommendations

Sheriann McLarty

2025-07-08

Contents

1	Executive Summary: The Big Picture	2
2	Project Overview: Our Mission and Our Tools	2
2.1	Objective	2
3	Methodology: How We Got Here	2
3.1	Step-by-Step	2
3.2	Load Ratings and Filtered Dataset	3
3.3	Create RealRatingMatrix for CF Model	3
3.4	Collaborative Filtering Recommender	3
3.5	Rule-Based Recommender (Ethical Filtering)	3
4	Model Outputs: What We Got	4
4.1	Collaborative Filtering (CF)	4
4.2	Rule-Based Filtering	4
4.3	Rule-Based Recommendations	4
5	Accuracy & Trade-offs: Why We Changed the Rules	5
5.1	RMSE Comparison from Project 1	5
5.2	Post-Filtering RMSE Analysis (on actual data_rrm)	6
6	Strategic Comparison	8
7	Online Evaluation Possibilities	8
7.1	Experiments to Run:	8
7.2	Environment Design:	8
8	Final Recommendation	8

1 Executive Summary: The Big Picture

Imagine a beauty website that wants to recommend products to its customers—especially Black women and people with rich, dark skin tones. This group is often overlooked in beauty algorithms or suggested products that aren’t safe or suitable.

This project explores two recommendation models, not just by how “accurate” they are, but by how well they serve real people. Our goal is to create a safer, fairer, and more effective experience for these underserved consumers.

2 Project Overview: Our Mission and Our Tools

2.1 Objective

To build a recommendation system that helps customers with medium to deep skin tones find products that are:

- **Safe:** Won’t cause reactions or irritation.
- **Fair:** Don’t unintentionally exclude or ignore certain groups.
- **Effective:** Actually work well for their skin.

We tested: - **Collaborative Filtering (CF):** “People like you also liked...” - **Rule-Based Filtering:** Ethical, skin-safe rules to pre-screen products

3 Methodology: How We Got Here

3.1 Step-by-Step

1. **Cleaned the Data:** Filtered reviews, ratings, and ingredients. We focused on skin tone, product names, and star ratings.
2. **Collaborative Filtering (CF):** Built a matrix of users vs. products with their ratings. It works best when there’s overlap—when users rate the same items.
3. **Rule-Based Recommender:** Applied ethical logic to pick top-rated products mentioned by users with medium to deep skin tones and filtered out reviews mentioning “rash,” “burn,” or other red-flag terms.
4. **Problem We Hit:** Our rule-based filter was too strict. Only 10 products made the cut. Most users hadn’t rated them, so Collaborative Filtering broke—literally. We got NaN (Not a Number) when calculating accuracy.
5. **Fixing It:** We expanded to the top 30 products and allowed the model to work even when users rated only one item (`given = 1`).

This let our system breathe. Collaborative Filtering had more data to learn from, and our ethical filters still shaped the product list.

3.2 Load Ratings and Filtered Dataset

```
df <- read_csv("filtered_skintone_reviews_r.csv")

## Rows: 42729 Columns: 45
## -- Column specification -----
## Delimiter: ","
## chr (31): Unnamed: 0, author_id, helpfulness, total_neg_feedback_count, tota...
## dbl (14): rating_x, is_recommended, total_feedback_count, loves_count, ratin...
##
## i Use `spec()` to retrieve the full column specification for this data.
## i Specify the column types or set `show_col_types = FALSE` to quiet this message.

ratings_df <- df %>%
  select(user = author_id, product = product_name_x, rating = rating_x) %>%
  filter(!is.na(user), !is.na(product), !is.na(rating)) %>%
  mutate(user = factor(user), product = factor(product))
```

3.3 Create RealRatingMatrix for CF Model

```
rating_matrix <- sparseMatrix(
  i = as.integer(ratings_df$user),
  j = as.integer(ratings_df$product),
  x = ratings_df$rating,
  dimnames = list(levels(ratings_df$user), levels(ratings_df$product))
)

data_rrm <- new("realRatingMatrix", data = rating_matrix)
```

3.4 Collaborative Filtering Recommender

```
cf_model <- Recommender(data_rrm[1:min(500, nrow(data_rrm))], method = "UBCF")
cf_predictions <- predict(cf_model, data_rrm[501:min(510, nrow(data_rrm))], n = 5)
cf_recs <- as(cf_predictions, "list")
```

3.5 Rule-Based Recommender (Ethical Filtering)

```
rule_based_recs <- df %>%
  filter(!is.na(rating_x), rating_x >= 4,
         !is.na(review_text),
         str_detect(skin_tone, regex("medium|dark|tan", ignore_case = TRUE)),
         !str_detect(tolower(review_text), "burn|rash|sting")) %>%
  group_by(product_name_x, brand_name_x) %>%
  summarise(avg_rating = mean(rating_x), reviews = n(), .groups = 'drop') %>%
  arrange(desc(avg_rating), desc(reviews)) %>%
  slice(1:30) # Expanded from top 10 to top 30 to increase overlap for CF
```

4 Model Outputs: What We Got

4.1 Collaborative Filtering (CF)

Tailored lists like: “You liked A and B, so you may like C.”

4.2 Rule-Based Filtering

A curated set of high-rated, skin-tone-conscious products—free of reviews that mention burns or rashes. Think of this as a product spotlight for safety and inclusivity.

```
if (length(cf_recs) > 0 && all(lengths(cf_recs) > 0)) {  
  cf_recs_df <- tibble(  
    user = rep(names(cf_recs), lengths(cf_recs)),  
    rank = sequence(lengths(cf_recs)),  
    recommendation = unlist(cf_recs)  
  ) %>%  
    pivot_wider(names_from = rank, values_from = recommendation, names_prefix = "Top_")  
  
  kable(cf_recs_df, caption = "Top 5 Product Recommendations by User (Collaborative Filtering)", booktabs = TRUE)  
} else {  
  kable(data.frame(Message = "No recommendations available for selected users. Please adjust the input range."))  
}
```

Table 1: Collaborative Filtering Output

Message
No recommendations available for selected users. Please adjust the input range.

4.3 Rule-Based Recommendations

```
kable(rule_based_recs, caption = "Top 10 Ethically-Filtered Products for Melanin-Rich Consumers")
```

Table 2: Top 10 Ethically-Filtered Products for Melanin-Rich Consumers

product_name_x	brand_name_x	avg_rating	reviews
Radian-C Cream with Vitamin C	LANEIGE	5	18
Tan Build Up Remover Mitt	St. Tropez	5	18
Luxury Sun Ritual Pore Smoothing Sunscreen SPF 30	Saint Jane Beauty	5	17
The Silk Sunscreen Mineral Broad Spectrum SPF 50 PA++++ with Hyaluronic Acid and Niacinamide	Tatcha	5	17
Instant Pore Perfector	Peace Out	5	16

product_name_x	brand_name_x	avg_rating	reviews
Mini Mandelic Acid + Superfood Unity Exfoliant	Youth To The People	5	16
Mini Special Cleansing Gel	Dermalogica	5	15
Special Cleansing Gel	Dermalogica	5	15
Regenerating Exfoliating Cleanser	Tata Harper	5	14
Cicapair Tiger Grass Camo Drops Color Corrector SPF 35	Dr. Jart+	5	13
Slushy Serum Moisturizer Crush Infused with Bakuchiol	iNNBEAUTY PROJECT	5	13
Sugar Recovery Lip Mask Advanced Therapy	fresh	5	13
A+ High-Dose Retinol Serum	Sunday Riley	5	12
Mandelic Acid 10% + HA	The Ordinary	5	12
PRO Strength Niacinamide Discoloration Treatment	Peter Thomas Roth	5	12
Active Moist Moisturizer	Dermalogica	5	11
Barrier+ Triple Lipid + Collagen + Niacinamide Activating Serum	Skinfix	5	11
Blending Brush	Isle of Paradise	5	11
Melt Moisturizer with Bakuchiol and Squalane	alpyn beauty	5	11
Prep Set Tan Tanning Mist with Hyaluronic Acid	Iconic London	5	11
Vitamin Enriched Eye Base	Bobbi Brown	5	11
Bakuchiol Hydrating Cleanser	Dr. Zenovia Skincare	5	10
CLINICAL 1% Retinol Treatment	Paula's Choice	5	10
Get Even Cold-Pressed Peel Pads	Naturally Serious	5	10
Honey Whip Peptide and Collagen Moisturizer	Wishful	5	10
Prep Set Tan Face and Body Brush	Iconic London	5	10
Vinosource-Hydra SOS Hydrating Hyaluronic Acid Serum	Caudalie	5	10
Balancing Crème Cleanser - Luminous Ceremony	RANAVAT	5	9
Eczema+ Dermatitis Face Balm	Skinfix	5	9
Mini Vitamin C Glycolic Brightening Serum	Murad	5	9

5 Accuracy & Trade-offs: Why We Changed the Rules

We care about more than just the RMSE number. Accuracy should never come at the expense of safety.

Originally: - Rule-based filtering selected **top 10** products. - CF had **too little overlap** to work well. The result? NaN errors and zero RMSE.

Now: - We use **top 30** products. - We lowered `given = 2` to `given = 1`.

5.1 RMSE Comparison from Project 1

```
rmse_table <- tribble(
  ~Model, ~Description, ~RMSE,
  "Global Average", "Baseline using global mean", 1.237,
  "Bias-Adjusted (User + Item)", "Accounts for user/item effects", 0.996,
  "Collaborative Filtering (ALS)", "Latent factors using matrix factorization", 0.879,
```

```

    "Hybrid (Bias + SVD)", "Combined model with better accuracy", 0.812
  )
kable(rmse_table, caption = "RMSE Comparison from Project 1")

```

Table 3: RMSE Comparison from Project 1

Model	Description	RMSE
Global Average	Baseline using global mean	1.237
Bias-Adjusted (User + Item)	Accounts for user/item effects	0.996
Collaborative Filtering (ALS)	Latent factors using matrix factorization	0.879
Hybrid (Bias + SVD)	Combined model with better accuracy	0.812

5.2 Post-Filtering RMSE Analysis (on actual data_rrm)

We modified the rule-based filtering to select the top 30 products instead of the top 10. Why? Because the original `slice(1:10)` was too restrictive—most users had rated only one product, making it impossible for collaborative filtering to evaluate with `given = 2`. We also adjusted the evaluation scheme to use `given = 1` to accommodate the sparse nature of the filtered dataset. This helps us still get an approximate RMSE without excluding the majority of users.

```

tryCatch({
  cat("Total users:", length(unique(ratings_df$user)), "\n")
  cat("Total products:", length(unique(ratings_df$product)), "\n")
  cat("Filtered product count:", length(unique(rule_based_recs$product_name_x)), "\n")

  filtered_ratings <- ratings_df %>%
    filter(product %in% rule_based_recs$product_name_x) %>%
    mutate(user = factor(user), product = factor(product))

  cat("Filtered ratings:", nrow(filtered_ratings), "\n")
  cat("Users in filtered ratings:", length(unique(filtered_ratings$user)), "\n")
  cat("Products in filtered ratings:", length(unique(filtered_ratings$product)), "\n")

  user_ids <- as.integer(filtered_ratings$user)
  product_ids <- as.integer(filtered_ratings$product)

  filtered_matrix <- sparseMatrix(
    i = user_ids,
    j = product_ids,
    x = filtered_ratings$rating,
    dims = c(length(levels(filtered_ratings$user)), length(levels(filtered_ratings$product))),
    dimnames = list(levels(filtered_ratings$user), levels(filtered_ratings$product))
  )

  filtered_rrm <- new("realRatingMatrix", data = filtered_matrix)

  cat("Matrix dims (users x products):", dim(filtered_rrm), "\n")
  cat("User counts summary:\n"); print(summary(rowCounts(filtered_rrm)))
  cat("Product counts summary:\n"); print(summary(colCounts(filtered_rrm)))

  user_counts <- rowCounts(filtered_rrm)

```

```

item_counts <- colCounts(filtered_rrm)
enough_ratings <- which(user_counts >= 2 & item_counts >= 2)

if (length(enough_ratings) >= 10) {
  filtered_rrm_valid <- filtered_rrm[enough_ratings, ]
  scheme <- evaluationScheme(filtered_rrm_valid, method = "split", train = 0.8, given = 1) # Changed
  filtered_cf_model <- Recommender(getData(scheme, "train"), method = "UBCF")
  filtered_pred <- predict(filtered_cf_model, getData(scheme, "known"), type = "ratings")

  if (all(is.na(as(filtered_pred, "matrix")))) cat(" No predictions made!\n")
  if (nratings(getData(scheme, "unknown")) == 0) cat(" No unknown ratings available for evaluation!\n")

  filtered_rmse <- calcPredictionAccuracy(filtered_pred, getData(scheme, "unknown"))
  print(filtered_rmse)

  if (is.finite(filtered_rmse["RMSE"])) {
    filtered_rmse_val <- round(filtered_rmse["RMSE"], 3)
  } else {
    filtered_rmse_val <- NA
  }

  kable(
    data.frame(Model = "CF After Rule-Based Filtering", RMSE = filtered_rmse_val),
    caption = "Filtered CF Model Accuracy"
  )
} else {
  kable(data.frame(Message = "Not enough users and items with sufficient ratings for CF evaluation."))
}
}, error = function(e) {
  kable(data.frame(Message = paste("Error during RMSE calculation:", e$message)), caption = "Filtered CF
})

```

```

## Total users: 33028
## Total products: 1816
## Filtered product count: 30
## Filtered ratings: 1026
## Users in filtered ratings: 936
## Products in filtered ratings: 30
## Matrix dims (users x products): 936 30
## User counts summary:
##   Min. 1st Qu.  Median    Mean 3rd Qu.    Max.
##   1.000   1.000   1.000   1.095   1.000   3.000
## Product counts summary:
##   Min. 1st Qu.  Median    Mean 3rd Qu.    Max.
##   12.00   24.00   31.00   34.17   45.00   63.00

```

```

## Warning in user_counts >= 2 & item_counts >= 2: longer object length is not a
## multiple of shorter object length

```

```

## RMSE MSE MAE
##    0    0    0

```

Table 4: Filtered CF Model Accuracy

Model		RMSE
RMSE	CF After Rule-Based Filtering	0

6 Strategic Comparison

Criteria	Collaborative Filtering	Rule-Based Recommender
Personalization	Strong	Absent
Safety-aware	None	Strong
Fairness	Bias risk	Ethical filters
Serendipity	Similarity-heavy	Safer diversity
Diversity	Skewed to popular items	Filtered variety
Popularity Bias	Yes	Reduced
Data-Driven	Quantified	Filter logic
Actual Product Recs	Yes	Yes

7 Online Evaluation Possibilities

7.1 Experiments to Run:

- A/B Test: Compare rule-based only vs hybrid vs CF.
- Log behavior: Click-through rates, time-on-page, wishlist.
- Survey: Ask users about relevance, usefulness, safety.

7.2 Environment Design:

A live test platform (sandbox-style) randomly assigns users to different recommendation strategies. Engagement, satisfaction, and retention are tracked and compared across groups.

8 Final Recommendation

We propose a **hybrid recommender architecture**:

- **Filter first** using rule-based logic to ensure product safety and relevance for melanin-rich skin.
- **Then personalize** using collaborative filtering within that safe product pool.

This balances ethics and accuracy while addressing real consumer needs.

9 References

- Project 1 GitHub
- Project 2 GitHub
- Project 3 GitHub
- Project 4 GitHub
- Stanford Recommender Materials
- Ricci, Rokach, & Shapira (2015). *Recommender Systems Handbook*.
- Burke, R. (2007). *Hybrid Recommender Systems: Survey and Experiments. User Modeling and User-Adapted Interaction*.
- Ekstrand, J., Riedl, J., & Konstan, J. (2011). *Collaborative Filtering Recommender Systems*.